Docker Volume and network

Class no:3

Docker Volumes:

In Docker, containers are **ephemeral** (temporary). When a container is deleted, any data created or modified inside it is lost forever. To persist data and share it between containers, Docker provides **Volumes**.

1. Why Do We Need Volumes?

- **Data Persistence:** Keeps data alive even if the container is stopped, crashed, or deleted.
- **Performance:** Writing to a container's writable layer requires a storage driver, which reduces I/O performance. Volumes bypass this layer and write directly to the host storage.
- **Data Sharing:** Multiple containers can attach to the same volume simultaneously to share data.
- **Decoupling:** Separates the application code/runtime from the data it generates.

2. Types of Data Storage in Docker

Docker offers three main ways to store data on the host machine:

Storage Type	Host Location	Best Used For
Volumes	Managed by Docker (/var/lib/docker/volumes/ on Linux).	Persisting database data, production storage, sharing data between containers.
Bind Mounts	Anywhere on the host system (e.g., /home/user/project).	Source code caching during development, sharing host configuration files.
tmpfs Mounts	Stored only in the host's **system memory** (RAM).	Storing sensitive data (passwords/keys) or high-frequency temporary data that shouldn't be written to disk.

3. Core Docker Volume Commands

- **Create a volume:** docker volume create volume_name
- **List all volumes:** docker volume ls
- **Inspect a volume (find its physical path and metadata):** docker volume inspect volume_name
- **Remove a specific volume:** docker volume rm volume_name
- **Prune (delete all unused volumes):** docker volume prune

Introduction to Docker Networking

In Docker, isolation is a core feature. By default, containers cannot talk to each other or the outside world unless you explicitly allow them to. **Docker Networking** is the subsystem that enables containers to communicate with the host machine, other containers, and external networks securely.

Docker uses **Network Drivers** to provide different types of networking capabilities.

The Core Docker Network Drivers

Docker comes with several built-in network drivers. Choosing the right one depends on your application's architecture.

1. Bridge Network (Default)

- **How it works:** When you install Docker, a default bridge network (usually named bridge) is created automatically. It creates a virtual private network inside your host machine. Every container connected to this network gets its own internal IP address.
- **Communication:** Containers on the same bridge network can talk to each other using their IP addresses. If you create a *custom* user-defined bridge network, containers can also talk to each other using their **container names** (built-in DNS resolution).
- **Use Case:** Best for standard applications running on a single host where multiple containers need to communicate (e.g., a web app talking to a local database).

2. Host Network

- **How it works:** This driver removes the network isolation between the container and the Docker host. The container shares the host's networking namespace directly.
- **Communication:** If a container runs an application on port 80, that application is immediately available on port 80 of the host's physical IP address. You don't need to publish ports using -p.
- **Use Case:** Best for optimizing performance (as there is no routing overhead) or handling large volumes of data on a single host.

Summary Comparison Table

Network Driver	Isolation Level	Scope	DNS Resolution?	Typical Scenario
Bridge	Private to Host	Single Host	Yes *(User-defined only)*	Standard multi-container apps on one machine.
Host	None (Uses Host)	Single Host	No	Maximizing network performance / low latency.

Core Docker Network Commands

Here is a quick cheat sheet for managing networks via the command line:

- **List all networks:** docker network ls
- **Create a user-defined bridge network:** docker network create my_custom_network
- **Run a container on a specific network:** docker run -d --name web_server --network my_custom_network nginx
- **Connect an existing container to a network:** docker network connect my_custom_network existing_container
- **Disconnect a container from a network:** docker network disconnect my_custom_network existing_container
- **Inspect a network (to see connected containers and IPs):** docker network inspect my_custom_network
- **Remove an unused network:** docker network rm my_custom_network

Hands on creating a docker volume and attaching it with a container to take backup

Step no:1 Create a named Docker volume to persist data

docker volume create myvolume

Pull the latest Apache HTTP Server (httpd) image from Docker Hub

docker pull httpd:latest

```
root@ip-172-31-4-169:/home/ubuntu# docker volume create myvolume
myvolume
root@ip-172-31-4-169:/home/ubuntu# docker pull httpd:latest
latest: Pulling from library/httpd
ed804d184361: Pull complete
5b4d6ff92fc4: Pull complete
f086d7c0b862: Pull complete
f60b6f23733a: Pull complete
c3f994ae0cac: Pull complete
4f4fb700ef54: Pull complete
698ed343ab91: Download complete
d2499f6f57b0: Download complete
Digest: sha256:2a7deaaaf357261a1dffcb8fc725f4aaba2af95fbde1a40a68
Status: Downloaded newer image for httpd:latest
docker.io/library/httpd:latest
```

Step no:2 Run a container named '**conatiner1**' in detached interactive mode (-itd),

mapping host port 8080 to container port 80, and attaching 'myvolume' to '/app/data'

docker run -itd --name conatiner1 -p 8080:80 -v myvolume:/app/data httpd:latest

Verify that the container is up and running properly

docker ps

```
docker.io/library/httpd:latest
root@ip-172-31-4-169:/home/ubuntu# docker run -itd --name conatiner1 -p 8080:80 -v myvolume:/app/data httpd:latest
08c400de5ddde36ef7a733e73dd34c5c9b4a633787e63b51d2e064c13e6a14cb
root@ip-172-31-4-169:/home/ubuntu# docker ps
CONTAINER ID   IMAGE          COMMAND              CREATED         STATUS         PORTS                                     NAMES
08c400de5ddd   httpd:latest   "httpd-foreground"   6 seconds ago   Up 6 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   conatiner1
```

Step no:3 Access the running container's bash terminal interactively

docker exec -it conatiner1 /bin/bash

Inside container1 terminal: List the directory contents to explore the Apache file structure: **ls**

Change directory to the attached volume's mount point: **cd /app/data**

Verify the directory is empty initially: **ls**

Create three sample files inside the mounted volume directory: **touch file1 file2 file3**

Verify that the files were successfully created: **ls**

Exit from the container's interactive shell back to the host machine: **exit**

```
root@ip-172-31-4-169:/home/ubuntu# docker exec -it conatiner1 /bin/bash
root@08c400de5ddd:/usr/local/apache2# ls
bin  build  cgi-bin  conf  error  htdocs  icons  include  logs  modules
root@08c400de5ddd:/usr/local/apache2# cd /app/data
root@08c400de5ddd:/app/data# ls
root@08c400de5ddd:/app/data# touch file1 file2 file3
root@08c400de5ddd:/app/data# ls
file1  file2  file3
root@08c400de5ddd:/app/data# exit
```

Step no:4 Verifying Data Persistence on the Host Machine:

Navigate to the default host directory where Docker manages local volumes: **cd /var/lib/docker/volumes**

List the directory contents to find '**myvolume**': **ls**

Change directory into your specific volume folder: **cd myvolume**

List contents to see the '**_data**' directory where the filesystem resides: **ls**

Navigate inside the actual data storage directory: **cd _data/**

Confirm that **'file1', 'file2', and 'file3'** created inside the container exist on the host system: **ls**

```
root@ip-172-31-4-169:/home/ubuntu# cd /var/lib/docker/volumes
root@ip-172-31-4-169:/var/lib/docker/volumes# ls
backingFsBlockDev  metadata.db  myvolume
root@ip-172-31-4-169:/var/lib/docker/volumes# cd myvolume
root@ip-172-31-4-169:/var/lib/docker/volumes/myvolume# ls
_data
root@ip-172-31-4-169:/var/lib/docker/volumes/myvolume# cd _data/
root@ip-172-31-4-169:/var/lib/docker/volumes/myvolume/_data# ls
file1  file2  file3
```

Hands-on Working with Bind Mounts to take backup for a container

Step no:1 Navigate back to the root/home environment on the host machine: **cd**

whoami

Ensure to be on the root or /home/ubuntu

Pull the latest Nginx web server image from Docker Hub: **docker pull nginx:latest**

```
root@ip-172-31-4-169:/var/lib/docker/volumes/myvolume/_data# cd
root@ip-172-31-4-169:~# whoami
root
root@ip-172-31-4-169:~# docker pull nginx:latest
latest: Pulling from library/nginx
b4a248c845e5: Pull complete
7f8b1a2b17d8: Pull complete
```

Step no:2 Change directory to the host's home path: **cd /home/ubuntu**/

```
root@ip-172-31-4-169:~# cd /home
root@ip-172-31-4-169:/home# ls
ubuntu
root@ip-172-31-4-169:/home# cd ubuntu/
```

Create a dedicated directory on your host system to be used for the bind mount: **mkdir bindmountsfolder**

cd bindmountsfolder/

Print and copy the absolute path of the host folder: **pwd**

```
root@ip-172-31-4-169:/home/ubuntu# mkdir bindmountsfolder
root@ip-172-31-4-169:/home/ubuntu# cd bindmountsfolder/
root@ip-172-31-4-169:/home/ubuntu/bindmountsfolder# pwd
/home/ubuntu/bindmountsfolder
```

Step no:3 Run a new container named '**container2**' mapping host port 8081 to port 80,

bind-mounting the host's absolute path directly to '/app/data' inside Nginx

docker run -itd --name container2 -p 8081:80 -v /home/ubuntu/bindmountsfolder:/app/data nginx:latest

Check the running status of both container1 and container2: **docker ps**

```
root@ip-172-31-4-169:/home/ubuntu/bindmountsfolder# docker run -itd --name container2 -p 8081:80 -v /home/ubuntu/bindmountsfolder:/app/data nginx:latest
f7b8619bb666ed0881524dbd26da528af00d3a78b0ac6ac372d4987762b3e747
root@ip-172-31-4-169:/home/ubuntu/bindmountsfolder# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                                                  NAMES
f7b8619bb666   nginx:latest   "/docker-entrypoint.…"   9 seconds ago    Up 9 seconds    0.0.0.0:8081->80/tcp, [::]:8081->80/tcp                container2
08c400de5ddd   httpd:latest   "httpd-foreground"       10 minutes ago   Up 10 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp                conatiner1
```

Step no:4 Execute an interactive shell inside **container2**

docker exec -it container2 /bin/bash

Change directory to the bind-mounted path: **cd /app/data → ls**

Create a new folder named '**newnote**' within the mount point

mkdir newnote

cd newnote/

Generate files inside the new directory**: touch file4 file5 file6**

Exit from the container back to your host terminal: **exit**

```
root@ip-172-31-4-169:/home/ubuntu/bindmountsfolder# docker exec -it container2 /bin/bash
root@f7b8619bb666:/# cd /app/data
root@f7b8619bb666:/app/data# ls
root@f7b8619bb666:/app/data# mkdir newnote
root@f7b8619bb666:/app/data# cd newnote/
root@f7b8619bb666:/app/data/newnote# touch file4 file5 file6
root@f7b8619bb666:/app/data/newnote# exit
exit
```

Step no:5 On the host machine terminal (**inside /home/ubuntu/bindmountsfolder**):

Verify that the folder '**newnote**' created inside the container is immediately visible on the host: **ls**

cd newnote/

Confirm that **'file4', 'file5', and 'file6'** exist natively on the host filesystem: **ls**

```
root@ip-172-31-4-169:/home/ubuntu/bindmountsfolder# ls
newnote
root@ip-172-31-4-169:/home/ubuntu/bindmountsfolder# cd newnote/
root@ip-172-31-4-169:/home/ubuntu/bindmountsfolder/newnote# ls
file4  file5  file6
```

Hands-on with Docker Networking

Step no:1 Create a custom user-defined bridge network named '**mynetwork**': **docker network create mynetwork**

List all available Docker networks to confirm '**mynetwork**' is created with the bridge driver: **docker network ls**

```
root@ip-172-31-4-169:/home/ubuntu# docker network create mynetwork
49a54b1dabc450850a9c4656d964b46ee2a58932a9f927624f001ade9a96418f
root@ip-172-31-4-169:/home/ubuntu# docker network ls
NETWORK ID     NAME        DRIVER    SCOPE
fbf16d646073   bridge      bridge    local
cd3ec9c6aa37   host        host      local
49a54b1dabc4   mynetwork   bridge    local
e9a7a33a9b2f   none        null      local
```

Step no:2 Connect the existing container '**conatiner1**' to the new '**mynetwork**' interface

docker network connect mynetwork 08c400de5ddd

```
root@ip-172-31-4-169:/home/ubuntu# docker network connect mynetwork 08c400de5ddd
root@ip-172-31-4-169:/home/ubuntu# docker inspect 08c400de5ddd
[
```

Inspect container1 to verify its network configuration settings, IP allocation, and gateway

docker inspect 08c400de5ddd

```
},
"mynetwork": {
    "IPAMConfig": {
        "IPv4Address": "",
        "IPv6Address": ""
    },
    "Links": null,
    "Aliases": [],
    "DriverOpts": {},
    "GwPriority": 0,
    "NetworkID": "49a54b1dabc450850a9
    "EndpointID": "7637c1c76b0f9d0b10
    "Gateway": "172.18.0.1",
    "IPAddress": "172.18.0.2",
```

Step no:3 Connect the second container '**container2**' to the exact same network:

docker network connect mynetwork container2

```
root@ip-172-31-4-169:/home/ubuntu# docker network connect mynetwork container2
```

Open an interactive bash environment inside container2 to check its configuration: **docker exec -it container2 /bin/bash**

```
},
"mynetwork": {
    "IPAMConfig": {
        "IPv4Address": "",
        "IPv6Address": ""
    },
    "Links": null,
    "Aliases": [],
    "DriverOpts": {},
    "GwPriority": 0,
    "NetworkID": "49a54b1dabc450
    "EndpointID": "107d5e76ab85a
    "Gateway": "172.18.0.1",
    "IPAddress": "172.18.0.3",
```

Step no:4 Inside container2 terminal:

Review the running OS platform configuration details

cat /etc/os-release

```
root@ip-172-31-4-169:/home/ubuntu# docker exec -it container2 /bin/bash
root@f7b8619bb666:/# cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION="13 (trixie)"
VERSION_CODENAME=trixie
DEBIAN_VERSION_FULL=13.5
ID=debian
HOME_URL="https://www.debian.org/"
```

Step no:5 Synchronize the package repository index lists to prepare for network utility installations: **apt update -y**

Similarly **update nginx & install iputils-ping**

```
root@f7b8619bb666:/# apt update -y
Get:1 http://deb.debian.org/debian tri
Get:2 http://deb.debian.org/debian tri
Get:3 http://deb.debian.org/debian-sec
Get:4 http://deb.debian.org/debian tri
Get:5 http://deb.debian.org/debian tri
Get:6 http://deb.debian.org/debian-sec
Fetched 10.1 MB in 1s (14.4 MB/s)
```

```
root@f7b8619bb666:/# apt install iputils-ping -y
Installing:
  iputils-ping

Installing dependencies:
  linux-sysctl-defaults

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 5
  Download size: 57.0 kB
  Space needed: 211 kB / 27.0 GB available
```

Ping the ip address of the container1 in conatiner2: **ping 172.18.0.2**

```
root@f7b8619bb666:/# ping 172.18.0.2
PING 172.18.0.2 (172.18.0.2) 56(84) bytes of data.
64 bytes from 172.18.0.2: icmp_seq=1 ttl=64 time=0.078 ms
64 bytes from 172.18.0.2: icmp_seq=2 ttl=64 time=0.057 ms
64 bytes from 172.18.0.2: icmp_seq=3 ttl=64 time=0.059 ms
```

Step no:6 Inside container1 terminal:

Review the running OS platform configuration details

cat /etc/os-release

```
root@ip-172-31-4-169:/home/ubuntu# docker exec -it conatiner1 /bin/bash
```

```
root@08c400de5ddd:/usr/local/apache2# apt update -y
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64
Fetched 9944 kB in 1s (14.1 MB/s)
4 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@08c400de5ddd:/usr/local/apache2# apt install iputils-ping -y
Installing:
  iputils-ping
```

Ping the **ip** address of the container1 in conatiner2: **ping 172.18.0.3**

```
root@08c400de5ddd:/usr/local/apache2# ping 172.18.0.3
PING 172.18.0.3 (172.18.0.3) 56(84) bytes of data.
64 bytes from 172.18.0.3: icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from 172.18.0.3: icmp_seq=2 ttl=64 time=0.060 ms
64 bytes from 172.18.0.3: icmp_seq=3 ttl=64 time=0.058 ms
```

Points to remember:

In order to remove the volume, first stop the containers. Only then we can remove the volume.

```
CONTAINER ID   IMAGE          COMMAND                  CREATED
f7b8619bb666   nginx:latest   "/docker-entrypoint.…"   20 minutes ago
08c400de5ddd   httpd:latest   "httpd-foreground"       31 minutes ago
root@ip-172-31-4-169:/home/ubuntu# docker stop conatiner1 container2
conatiner1
container2
```

Get inside of the docker location and remove the volume. Similarly for network as well.

```
root@ip-172-31-4-169:/home/ubuntu# cd /var/lib/docker/volumes
root@ip-172-31-4-169:/var/lib/docker/volumes# ls
backingFsBlockDev  metadata.db  myvolume
root@ip-172-31-4-169:/var/lib/docker/volumes# rm myvolume
rm: cannot remove 'myvolume': Is a directory
root@ip-172-31-4-169:/var/lib/docker/volumes# rm -rf myvolume
root@ip-172-31-4-169:/var/lib/docker/volumes# ls
backingFsBlockDev  metadata.db
```